

JPA - Java Persistence API

Programação Distribuída / José Marinho – 2025/2026

Introdução

- Java Persistence API
- Mapeamento entre objetos e bases de dados (BD) relacionais
- Interação simplificada com as bases de dados
 - A API JPA oferece um nível de abstração superior que abstrai os programadores da complexidade relativa do JDBC e da linguagem SQL pura

2

Programação Distribuída / José Marinho – 2025/2026

Conceitos-chave

- **Object-Relational Mapping (ORM)**
 - Atua como uma ponte entre os objetos Java (paradigma orientado a objetos) e as BD relacionais (paradigma relacional)
- **Entidade**
 - Classe que representa uma tabela numa BD relacional
 - Uma instância corresponde a uma linha (registo) na tabela
 - Cada entidade deve possuir um identificador/chave único (pode ser simples ou composto)
 - Por omissão, os nomes dos atributos da classe correspondem aos nomes das colunas da tabela na BD

3

Programação Distribuída / José Marinho – 2025/2026

Conceitos-chave

- **JPQL (Java Persistence Query Language)**
 - Linguagem de consulta orientada a objetos para manipular entidades JPA em vez de consultar tabelas via SQL diretamente
- **Entidades e relacionamentos**
 - Definem como as classes Java (entidades) mapeiam em tabelas da BD e como estas se relacionam (ex.: @OneToMany)
- **Transações**
 - Garantem a integridade dos dados através de operações que respeitam as propriedades ACID (*Atomicity*, *Consistency*, *Isolation*, e *Durability*)

4

Programação Distribuída / José Marinho – 2025/2026

Conceitos-chave

- **Anotações**
 - Definem detalhes de mapeamento e outras configurações na classe Java (ex.: `@Entity` e `@Column`)
- **Independência do sistema**
 - Facilita a portabilidade entre diferentes sistemas de BD (MySQL, PostgreSQL, Oracle, etc.) sem alterar o código da aplicação

5

Programação Distribuída / José Marinho – 2025/2026

Exemplos de definição de uma entidade

- **Mapeamento básico**

```
import jakarta.persistence.*;

@Entity
public class Customer { //Nome da tabela na BD

    @Id
    private Long id; //Chave primária
    private String firstName;
    private String lastName;

    protected Customer() {}

    public Customer(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
    //...
}
```

6

Programação Distribuída / José Marinho – 2025/2026

Exemplos de definição de uma entidade

- Neste exemplo básico:
 - `@Entity` marca a classe para ser gerida pelo JPA. Por omissão, o nome da tabela será o nome da classe (`Customer`)
 - `@Id` designa o atributo `id` como a chave primária da tabela
 - Um construtor vazio (`public` ou `protected`) é obrigatório para o JPA

7

Programação Distribuída / José Marinho – 2025/2026

Exemplos de definição de uma entidade

- Mapeamento “personalizado”

```
import jakarta.persistence.*;

@Entity (name = "Empregado") //Nome da tabela na BD
public class Customer {

    @Id @Column(name = "CUSTOMER_ID", nullable = false)
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id; //Chave primária

    @Column(name = "NAME_1", nullable = false, LENGTH = 100)
    private String firstName;

    private String lastName;

    protected Customer() {}

    //...
```

8

Programação Distribuída / José Marinho – 2025/2026

Exemplos de definição de uma entidade

- Neste exemplo “personalizado”:
 - `@Entity(name = "Empregado")` muda o nome da tabela na base de dados para "Empregado"
 - `@Column(name = "...", nullable = ...)` personaliza o mapeamento do atributo (nome da coluna, se pode ou não ser nula, etc.)
 - `@GeneratedValue(strategy=...)` define como a chave primária é gerada (ex.: auto-incremento)

9

Programação Distribuída / José Marinho – 2025/2026

Exemplo de utilização numa aplicação com a *framework* Spring

```
//...
//Inserção de linhas/registos na tabela da BD
repository.save(new Customer("Jack", "Bauer"));
repository.save(new Customer("Chloe", "O'Brian"));
repository.save(new Customer("Kim", "Bauer"));
//...
```

- Neste exemplo:
 - O Spring Data JPA gere a conexão, a transação e a geração de comandos SQL (INSERT ou UPDATE) automaticamente
 - A interação com a BD é abstraída por um repositório (ex.: `CrudRepository`)
 - O método `repository.save()` é usado tanto para inserir novos registos como para atualizar registos existentes

10

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- Considere que se pretende aceder, numa BD, a uma tabela, com chave primária composta, com a estrutura definida pela instrução SQL seguinte:

```
CREATE TABLE pi_worker (  
    address VARCHAR(100) NOT NULL,  
    port INT NOT NULL,  
    timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON  
        UPDATE CURRENT_TIMESTAMP,  
    PRIMARY KEY (address, port)  
);
```

11

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- Como a chave primária é formada por mais de uma coluna (abordagem de chave composta):
 - Define-se uma classe que representa a chave composta (ex.: `PiWorkerPrimaryKey`)
 - Recorre-se à anotação `@IdClass(PiWorkerPrimaryKey.class)` para informar o JPA para usar a classe `PiWorkerPrimaryKey` para gerir a identidade
 - Os atributos que compõem a chave (`address` e `port`) são ambos marcados com a anotação `@Id`

12

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

```
public class PiWorkerPrimaryKey implements Serializable {
    private String address;
    private int port;

    public PiWorkerPrimaryKey() {}
    public PiWorkerPrimaryKey(String address, int port) {
        this.address = address; this.port = port;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PiWorkerPrimaryKey otherKey = (PiWorkerPrimaryKey) o;
        return port == otherKey.port && address.equals(otherKey.address);
    }

    @Override
    public int hashCode() { return Objects.hash(address, port); }
}
```

13

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- A classe identidade (PiWorkerPrimaryKey), que representa a chave composta, deve:
 - Implementar a interface `java.io.Serializable` para permitir a manipulação da chave em processos que envolvam aspetos como a persistência de dados, a utilização de caches e a comunicação em sistemas distribuídos.
 - Ter os mesmos atributos (e tipos) que as colunas da entidade marcadas com `@Id`
 - Ter os métodos `equals()` e `hashCode()` corretamente definidos para que o JPA possa gerir e comparar as identidades das entidades corretamente

14

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

```
import jakarta.persistence.*;
import java.sql.Timestamp;

@Entity (name = "pi_worker") //The table has a distinct name from the class
@IdClass(PiWorkerPrimaryKey.class) //Defines the composite primary key
public class PiWorker {
    @Id
    private String address;
    @Id
    private int port;
    //@Column(nullable = false)
    private Timestamp timestamp = null;

    protected PiWorker() {}

    public PiWorker(String address, int port) {
        this.address = address; this.port = port;
    }
    //... Getters, setters, equals, hashCode, toString, etc.
}
```

15

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

```
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface PiWorkerRepository extends CrudRepository<PiWorker,
                                                    PiWorkerPrimaryKey> {

    Iterable<PiWorker> findByAddress(String address);
    Iterable<PiWorker> findByPort(int port);
}
```

- @Repository
 - Anotação que identifica uma interface como sendo responsável por aceder à BD, permitindo operações de persistência automatizadas

16

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- Interface `CrudRepository`
 - CRUD – *Create, Read, Update e Delete*
 - Fornece métodos para realizar operações comuns de persistência:
 - `save()` - insere ou atualiza uma entidade (registro)
 - `findById()` - devolve uma entidade identificada pelo seu ID (chave)
 - `findAll()` - lista todos as entidades existentes
 - `delete()` - apaga uma entidade
 - `count()` - devolve o número de entidades existentes
 - `existsById()` - verifica se existe uma entidade com um determinado ID (chave)

17

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- Quando se estende a interface `CrudRepository`, especifica-se o tipo da entidade (`PiWorker`) e da sua chave primária (`PiWorkerPrimaryKey`)
- *Queries* derivadas
 - Podem ser definidos métodos de consulta adicionais
 - Neste exemplo: `findByAddress()` e `findByPort()`
 - O Spring Data JPA gera automaticamente os comandos SQL correspondentes (ex.: ... WHERE address= ...) com base no nome destes métodos

18

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

```
//... imports ...

@Service
public class PiWorkerService {

    @Autowired
    private PiWorkerRepository piWorkerRepository;

    public Iterable<PiWorker> getPiWorkers(String address, Integer port) {
        if(address!=null) {
            return piWorkerRepository.findByAddress(address);
        } else if(port!=null) {
            return piWorkerRepository.findByPort(port);
        } else {
            return piWorkerRepository.findAll();
        }
    }

    . . .
}
```

19

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- A classe `PiWorkersService`, com anotação `@Service`:
 - Contém a lógica de negócio da aplicação
 - Utiliza um `piWorkerRepository` (injetado via `@Autowired`) para aceder aos dados
 - O método `getPiWorkers()` obtém *workers* com base em diferentes filtros (endereço ou porto) ou sem filtros, invocando os métodos correspondentes do repositório (`findByAddress()`, `findByPort()` e `findAll()`)

20

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

```
public PiWorker getPiWorker(String address, Integer port) {  
  
    Optional<PiWorker> piWorkerOpt = piWorkerRepository.findById(  
        new PiWorkerPrimaryKey(address, port));  
  
    //return piWorkerOpt.isPresent() ? piWorkerOpt.get():null;  
    return piWorkerOpt.orElse(null);  
}  
  
public boolean addPiWorker(PiWorker piWorker) {  
  
    if(piWorkerRepository.existsById(new PiWorkerPrimaryKey(  
        piWorker.getAddress(), piWorker.getPort()))) {  
        return false;  
    }  
    piWorker.setTimestamp(null);  
    piWorkerRepository.save(piWorker);  
  
    return true;  
}
```

21

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- `getPiWorker()`
 - Procura pelo ID (chave)
 - É necessário instanciar a classe da chave primária composta (`PiWorkerPrimaryKey`) e passá-la ao método `findById()` do repositório
- `addPiWorker()`
 - Verificaca a não existência (`existsById()`) antes de guardar a nova entidade com `save()`
 - `setTimestamp(null)` força a BD a usar o valor por omissão (`CURRENT_TIMESTAMP`)

22

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

```
public PiWorker deletePiWorker(String address, int port) {  
  
    PiWorkerPrimaryKey id = new PiWorkerPrimaryKey(address, port);  
  
    if(piWorkerRepository.findById(id).isEmpty())  
        return null;  
  
    PiWorker worker = piWorkerRepository.findById(id).get();  
    piWorkerRepository.deleteById(id);  
  
    return worker;  
}  
} // PiWorkersService
```

23

Programação Distribuída / José Marinho – 2025/2026

Exemplo 1: Tabela “pi_worker”

- deleteWorker()
 - Apaga um registo, começando por criar uma instância da chave primária (PiWorkerPrimaryKey)
 - Verifica se o registo existe (findById().isEmpty())
 - O método deleteById() do repositório (PiWorkerRepository) é usado para remover a entidade da base de dados, utilizando a chave primária composta

24

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

- Considere que se pretende aceder, numa BD, a uma tabela com a estrutura definida pela seguinte instrução SQL (chave primária simples):

```
CREATE TABLE contact (  
    id INTEGER NOT NULL PRIMARY KEY,  
    email VARCHAR(255),  
    name VARCHAR(255),  
    phone_number VARCHAR(255)  
);
```

25

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

```
import jakarta.persistence.*;  
  
@Entity  
public class Contact {  
    @Id  
    @GeneratedValue(strategy = GenerationType.AUTO)  
    private Integer id;  
    private String name;  
    private String email;  
    private String phoneNumber;  
  
    //... setters, getters, etc. ...  
}
```

26

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

- Classe `Contact`
 - Mapeamento mais comum, com uma chave primária simples (`id`) de tipo `Integer`
 - `@GeneratedValue(strategy = GenerationType.AUTO)` indica ao JPA para delegar a geração do ID à BD (ex.: auto-incremento)
 - O JPA mapeia `phoneNumber` para `phone_number` automaticamente

27

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import pt.isec.pdjpa.model.Contact;
import java.util.List;

@Repository
public interface ContactRepository extends JpaRepository<Contact, Integer> {
    List<Contact> findByNameLike(String name);
    List<Contact> findByEmail(String email);
    List<Contact> findByPhoneNumber(String phoneNumber);

    @Query("SELECT c FROM Contact c WHERE c.phoneNumber LIKE '+351%'")
    List<Contact> findPortuguesePhoneNumbers();
}
```

28

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

- Interface `ContactRepository`
 - Estende a interface `JpaRepository`, que é uma extensão de `CrudRepository` com mais funcionalidades (ex.: `flush()`, `deleteInBatch()` e `deleteAllInBatch()`)
 - *Queries* Derivadas: `findByNameLike()`, `findByEmail()` e `findByPhoneNumber()`
 - A anotação `@Query` permite escrever consultas personalizadas em JPQL (*Java Persistence Query Language*)

29

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

```
//... imports ...

@Service
public class ContactService {
    private ContactRepository contactRepository;

    @Autowired
    public ContactService(ContactRepository contactRepository) {
        this.contactRepository = contactRepository;
    }

    public Contact createContact(Contact c) {
        c.setId(null);
        return contactRepository.save(c);
    }
}
```

30

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

- Injeção de dependência
 - A anotação `@Autowired` no construtor é uma forma de injeção de dependência no Spring
 - O Spring irá fornecer (injetar) automaticamente uma instância de `ContactRepository`
- Forçar o `id` de `c` a `null` garante que o método `save()` do JPA entenda esta operação como uma inserção (INSERT) e não uma atualização, permitindo que a BD gere um novo ID (auto-incremento)

31

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

```
public Contact updateContact(Integer id, Contact c) {  
    Optional<Contact> curContactOpt = contactRepository.findById(id);  
  
    if(curContactOpt.isPresent()) {  
        Contact curContact = curContactOpt.get();  
        curContact.setName(c.getName());  
        curContact.setEmail(c.getEmail());  
        curContact.setPhoneNumber(c.getPhoneNumber());  
        return contactRepository.save(curContact);  
    } else {  
        return null;  
    }  
}
```

32

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

```
public Contact deleteContact(Integer id) {  
    Optional<Contact> curContactOpt = contactRepository.findById(id);  
  
    if(curContactOpt.isPresent()) {  
        Contact curContact = curContactOpt.get();  
        contactRepository.deleteById(id);  
        return curContact;  
    } else {  
        return null;  
    }  
}  
  
public List<Contact> getAllContacts() {  
    return contactRepository.findAll();  
}  
  
public List<Contact> getContactsByName(String name) {  
    return contactRepository.findByNameLike("%" + name + "%");  
}
```

33

Programação Distribuída / José Marinho – 2025/2026

Exemplo 2: Tabela “contact”

```
public List<Contact> getContactsByEmail(String email) {  
    return contactRepository.findByEmail(email);  
}  
  
public List<Contact> getContactsByPhoneNumber(String phoneNumber) {  
    return contactRepository.findByPhoneNumber(phoneNumber);  
}  
  
public List<Contact> getAllPortugueseContacts() {  
    return contactRepository.findPortuguesePhoneNumbers();  
}  
}
```

34

Programação Distribuída / José Marinho – 2025/2026

Configuração de um projeto Spring

- Com Maven e sistema de Gestão de BD MySQL

Dependências (ficheiro pom.xml)

```
<!-- MySQL driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-jc</artifactId>
</dependency>

<!-- Spring Data JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Ficheiro application.properties

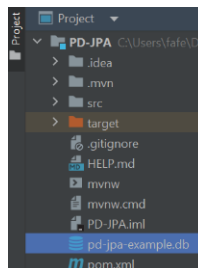
```
spring.jpa.hibernate.ddl-auto=none
spring.datasource.url=jdbc:mysql://sql7.freemysqlhosting.net:3306/sql7774
spring.datasource.username=sql7774
spring.datasource.password=wKsnTln
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

35

Programação Distribuída / José Marinho – 2025/2026

Configuração de um projeto Spring

- Com Maven e BD SQLite



Ficheiro application.properties

```
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:sqlite:pd-jpa-example.db
spring.datasource.driver-class-name=org.sqlite.JDBC
```

Dependências (ficheiro pom.xml)

```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
</dependency>

<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
</dependency>

<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-community-dialects</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-validator</artifactId>
</dependency>
```

36

Programação Distribuída / José Marinho – 2025/2026